In the Name of God

Course Project: Introduction to Machine Learning
Phase 1

Bayesian Neural Networks

Instructor
Dr. S. Amini

Department of Electrical Engineering
Sharif University of Technology

Spring 1404

Deadline:
Ordibehesht 25, 1404 at 23:55

Authors:
Mohammad Hossein Momeni, Amirhossein Naghdi, Hooman Zolfaghari,
Ghazal Hosseini, Yahya Tehrani, Amir Afzali, Borna Khodabandeh

Contents

1 Important Notes

Please carefully read the following points:

1. The two phases of this project will contribute a total of **4 points** to your final course grade.
2. After the final submission, there will be an in-person project defense session where you must present your code and outputs to the teaching assistants and answer their questions. **All group members must be fully familiar with every part of the project.** A single score will be assigned to the entire group.
3. All sections of both phases are mandatory. There will be no optional or bonus parts.
4. All simulations must be implemented using Python. You are allowed to use any libraries you have used in the assignments, such as `numpy`, `scipy`, and `pytorch`. However, **you must implement the algorithms yourself**, and you may not use libraries that already provide the algorithm implementations.
5. The project deliverables consist of both your code and a written report. The report should include answers to the questions, plots, figures, and conclusions. In the end, one .zip file containing all code files, and a .pdf report must be uploaded to the CW platform. Only one group member needs to submit the project.
6. If you use any external sources (books, articles, websites, etc.) to answer the questions, make sure to cite them properly.
7. **Any form of plagiarism will result in a zero for both parties involved.**
8. For any questions, please contact the project instructors via: @H_Theory, @amir_naghdi_2003, @mh_momeni, @YahyaTehrani, @GhLmu on Telegram.

Good luck!

In the Name of God

Introduction to Machine Learning (25737-2)

Course Project, Phase 1

Spring Semester 1404

Department of Electrical Engineering

Sharif University of Technology

Instructor: Dr. S.Amini

Due on Ordibehesht 25, 1404 at 23:55

2 Introduction

Bayesian Neural Networks (BNNs) combine the expressive power of deep neural networks with the principled framework of Bayesian inference, enabling models to quantify uncertainty in a meaningful way. Unlike traditional (frequentist) neural networks that provide point estimates for weights, BNNs treat weights as probability distributions, allowing them to model epistemic uncertainty.

Despite their advantages, BNNs face notable limitations. Exact Bayesian inference in neural networks is computationally intractable due to the high dimensionality of parameter space, necessitating approximate inference methods like Variational Inference or Markov Chain Monte Carlo (MCMC). These approximations introduce trade-offs between computational cost and predictive accuracy. Furthermore, implementing BNNs often involves complex training procedures and tuning hyperparameters related to prior distributions and variational approximations.

This project explores theoretical distinctions between Bayesian and frequentist paradigms, with an emphasis on advanced derivations in parameter estimation, prediction, and uncertainty quantification.

3 Frequentist and Bayesian Approaches

In the frequentist framework, model parameters $\boldsymbol{\theta}$ are viewed as fixed but unknown. The goal is to obtain a point estimate of these parameters, typically using Maximum Likelihood Estimation (MLE). For an i.i.d. dataset $X = \{x^{(1)}, x^{(2)}, \ldots, x^{(n)}\}$, the likelihood function is defined as

$$L(\theta|X) = p(X|\theta), \tag{1}$$

and the MLE is given by

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \; p(X|\theta). \tag{2}$$

Since the logarithm is a monotonic function, maximizing the log-likelihood

$$\ell(\theta|X) = \sum_{i=1}^{n} \log p(x^{(i)}|\theta), \tag{3}$$

is equivalent to maximizing the likelihood.

Theory Question 1. Prove that for a log-likelihood function derived from independent and identically distributed observations, the Hessian matrix (second derivative) is the sum of individual Hessians. Discuss under what conditions this Hessian is negative definite, ensuring the concavity of the log-likelihood function.

For a Gaussian likelihood with mean $f(X;\theta)$ and variance σ^2, the likelihood for a single observation is

$$p(y|x,\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(x;\theta))^2}{2\sigma^2}\right). \tag{4}$$

Taking the logarithm leads to a negative log-likelihood that is proportional to the sum-of-squared errors.

Theory Question 2. Derive in detail how the negative log-likelihood for a Gaussian likelihood reduces to the sum-of-squared errors (SSE), and then show how the minimization of this SSE is equivalent to maximizing the likelihood. In your derivation, carefully state any assumptions made about the constants involved.

In the Bayesian approach, the parameters θ are treated as random variables with a prior distribution $p(\theta)$. When new data (X, y) is observed, Bayes' rule updates our beliefs:

$$p(\theta|X,y) = \frac{p(y|X,\theta)p(\theta)}{p(y)}, \tag{5}$$

where the evidence $p(y)$ is typically intractable. Thus, we consider the proportionality:

$$p(\theta|X,y) \propto p(y|X,\theta)p(\theta). \tag{6}$$

Theory Question 3. Starting from Bayes' theorem, rigorously derive the expression

$$p(\theta|X,y) \propto p(y|X,\theta)p(\theta),$$

and provide a discussion on the role of the normalization constant $p(y)$. Why does this constant not affect the optimization when determining the mode of the posterior?

Predictions in the Bayesian framework are made by integrating over the parameter uncertainty:

$$p(y^{(n+1)}|x^{(n+1)}, X, y) = \int p(y^{(n+1)}|x^{(n+1)}, \theta)\, p(\theta|X,y)\, d\theta. \tag{7}$$

Theory Question 4. Derive the posterior predictive distribution starting from the joint distribution $p(y^{(n+1)}, \theta|x^{(n+1)}, X, y)$. Explain how the integration over θ mathematically encapsulates uncertainty in the parameter estimates, and derive an expression for the predictive variance in the case of a conjugate Gaussian model.

4 Maximum a Posteriori (MAP) Estimation

MAP estimation finds the most probable parameters by maximizing the posterior:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta}\ p(\theta|X,y) = \arg\max_{\theta}\ \left[\log p(y|X,\theta) + \log p(\theta)\right]. \tag{8}$$

For a Gaussian prior

$$p(\theta) = \mathcal{N}(0, \alpha^{-1}I), \tag{9}$$

the log-prior is

$$\log p(\theta) = -\frac{\alpha}{2}\|\theta\|^2 + \text{constant}. \tag{10}$$

Thus, the MAP estimation becomes a regularized MLE, introducing an L_2 penalty.

Theory Question 5. Starting from the Gaussian prior $p(\theta) = \mathcal{N}(0, \alpha^{-1}I)$, derive the MAP estimation formula in full detail. Show explicitly how the L_2 regularization term appears in the optimization problem. Then, discuss how the strength of the regularization depends on the hyperparameter α.

Theory Question 6. Prove that if the prior $p(\theta)$ is uniform, then the MAP estimate reduces exactly to the MLE. Include a discussion on the implications of this result for model complexity control.

5 Bayesian Neural Network

A Bayesian neural network is a probabilistic model that allows us to estimate uncertainty in predictions by representing the weights and biases of the network as probability distributions rather than fixed values. This allows us to incorporate prior knowledge about the weights and biases into the model, and update our beliefs about them as we observe data. Mathematically, a Bayesian neural network can be represented as follows: Given a set of input data x, we want to predict the corresponding output y. The neural network represents this relationship as a function $f(x;\theta)$, where θ are the weights and biases of the network. In a Bayesian neural network, we represent the weights and biases as probability distributions, so $f(x;\theta)$ becomes a probability distribution over possible outputs:

$$p(y|x,\mathcal{D}) = \int p(y|x,\theta)p(\theta|\mathcal{D})d\theta, \tag{11}$$

where $p(y|x,\theta)$ is the likelihood function, which gives the probability of observing y given x and θ , and $p(\theta|\mathcal{D})$ is the posterior distribution over the weights and biases given the observed data $\mathcal{D}$.

To make predictions, we use the posterior predictive distribution:

$$p(y^*|x^*,\mathcal{D}) = \int p(y^*|x^*,\theta)p(\theta|\mathcal{D})d\theta, \tag{12}$$

where x^* is a new input and y^* is the corresponding predicted output. To estimate the (intractable) posterior distribution $p(\theta|\mathcal{D})$, we can use either Markov Chain Monte Carlo (MCMC) or Variational Inference (VI).

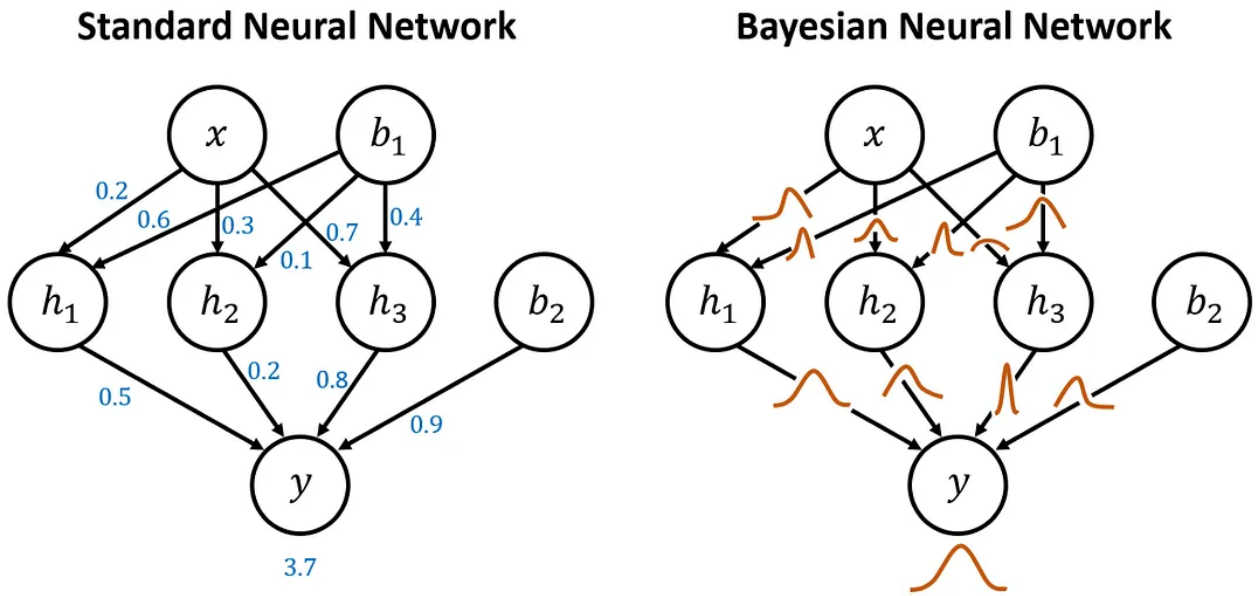

Figure 1: Comparison between a standard neural network and a Bayesian neural network (BNN)

6 Bayesian Inference and Monte Carlo Estimation

In Bayesian machine learning, one often needs to compute integrals over the posterior distribution. For example, the posterior predictive distribution can be written as:

$$p(y_{n+1} \mid x_{n+1},\mathcal{D}) = \int p(y_{n+1} \mid x_{n+1},\theta)\, p(\theta \mid \mathcal{D})\, d\theta, \tag{13}$$

where $\mathcal{D} = \{(x_1,y_1),\ldots,(x_n,y_n)\}$ is the dataset, and θ are the model parameters. This integral is typically intractable, but we can approximate it using Monte Carlo methods.

6.1 Monte Carlo Estimation

The idea of Monte Carlo estimation is to treat the integral as an expectation with respect to the posterior distribution $p(\theta \mid \mathcal{D})$. Let $f(\theta) = p(y_{n+1} \mid x_{n+1}, \theta)$, so that:

$$\mathbb{E}_{\theta \sim p(\theta|\mathcal{D})}[f(\theta)] = \int f(\theta)\, p(\theta|\mathcal{D})\, d\theta. \tag{14}$$

We can approximate this expectation by drawing samples $\{\theta^{(1)}, \ldots, \theta^{(N)}\}$ from $p(\theta|\mathcal{D})$, and compute the empirical average:

$$\hat{s}_N = \frac{1}{N} \sum_{i=1}^{N} f(\theta^{(i)}). \tag{15}$$

This estimator is unbiased, and under standard conditions (e.g., finite variance), it converges to the true value as $N \to \infty$ by the Law of Large Numbers:

$$\lim_{N \to \infty} \hat{s}_N = \mathbb{E}_\theta[f(\theta)]. \tag{16}$$

In addition, the Central Limit Theorem tells us:

$$\hat{s}_N \sim \mathcal{N}\left(\mathbb{E}[f(\theta)], \frac{\mathrm{Var}[f(\theta)]}{N}\right), \tag{17}$$

which allows us to construct confidence intervals around the estimate.

When direct sampling from $p(\theta \mid \mathcal{D})$ is not possible, MCMC methods such as Metropolis-Hastings or Hamiltonian Monte Carlo can be used to generate approximate samples.

6.2 A Simple Bayesian Neural Network

To illustrate Bayesian inference in practice, consider a simple BNN trained on a small regression dataset. The weights and biases of the network are treated as random variables, sampled from prior distributions (e.g., standard Gaussians).

A naïve way to approximate the posterior is to generate many networks by sampling their parameters from the prior, then accept or reject them based on how well they explain the data (likelihood). This is inefficient because very few samples are accepted, especially as the dataset grows.

Instead, a Monte Carlo approach using posterior samples improves this. For each sampled $\theta^{(i)}$, compute the network output $f(x_{n+1}; \theta^{(i)})$, and average over the results. The final prediction is:

$$\hat{y}_{n+1} = \frac{1}{N} \sum_{i=1}^{N} f(x_{n+1}, \theta^{(i)}). \tag{18}$$

This gives not only a point estimate but also a measure of uncertainty, since the variation among predictions reflects uncertainty in the model parameters.

Theory Question 7. Monte Carlo estimators rely on the Law of Large Numbers and the Central Limit Theorem. Explain the conditions required for these theorems to hold. What happens if the posterior distribution has heavy tails or infinite variance?

Theory Question 8. In the Bayesian Neural Network example, suppose only 6 out of 100,000 networks are accepted through rejection sampling. What does this tell you about the relationship between the prior and the likelihood? Why is rejection sampling inefficient in high-dimensional models?

7 Markov Chain Monte Carlo (MCMC)

Markov Chain Monte Carlo (MCMC) is a method for sampling from a target probability distribution $\mathbb{P}(\theta|D)$ by constructing a Markov chain. The chain is designed so that its stationary distribution[1] is the target distribution. By running the chain for enough iterations, the samples $\{\theta^{(0)}, \theta^{(1)}, \ldots, \theta^{(N)}\}$ converge to the desired distribution. MCMC allows for the approximation of integrals, expectations, and other properties of the target distribution without needing to compute the normalization constant, which is often difficult or impossible to obtain directly.

7.1 Markov Chain

A Markov chain is a sequence of random variables $\{X_t\}_{t\in\mathbb{N}}$ where the future state X_{t+1} depends only on the current state X_t and not on the past states $X_{t-1}, X_{t-2}, \ldots$. Formally:

$$\mathbb{P}(X_{t+1} = x_{t+1} \mid X_t = x_t, X_{t-1} = x_{t-1}, \ldots, X_0 = x_0) = \mathbb{P}(X_{t+1} = x_{t+1} \mid X_t = x_t). \tag{19}$$

The process is defined over a set of states $\mathcal{S}$, and the transitions between states are governed by a transition probability matrix P, where $P(x \to y) = \mathbb{P}(X_{t+1} = y \mid X_t = x)$ represents the probability of transitioning from state x to state y.

A Markov chain is said to be:

- **Irreducible** if it is possible to reach any state from any other state (i.e., for all $x, y \in \mathcal{S}$, there exists some t such that $P^t(x \to y) > 0$).
- **Aperiodic** if the greatest common divisor of the number of steps to return to any state is 1, meaning that the chain does not exhibit periodic behavior.
- **Positive recurrent** if, for every state $x \in \mathcal{S}$, the expected return time to x, denoted by $\mathbb{E}[\tau_x]$, is finite.

And finally, a Markov chain is said to have a **steady-state distribution** $\pi(x)$ if, as $t \to \infty$, the distribution of X_t converges to $\pi(x)$ (assume that $\pi(x) > 0$ for each state x). That is, the chain reaches equilibrium, and the probability of being in state x does not change with time:

$$\pi(x) = \sum_{y\in\mathcal{S}} \pi(y)P(y \to x). \tag{20}$$

If a Markov chain is ergodic (irreducible + aperiodic + positive recurrent), then the stationary distribution exists and is unique, and the chain converges to it — i.e., it is also the steady-state distribution. In such cases, $\pi(x)$ is the unique solution to the system of equations defined by the detailed balance condition or the stationary equation.

Theory Question 9. Prove that for an irreducible and aperiodic Markov chain, the chain converges to a unique stationary distribution $\pi(x)$, regardless of the initial distribution. Discuss the conditions under which convergence occurs and the implications for the chain's long-term behavior.

7.2 Metropolis-Hastings Algorithm

The most fundamental and widely used MCMC algorithm is the Metropolis-Hastings (MH) algorithm. The basic idea behind the MH algorithm is to generate a new state θ^* from a proposal distribution $q(\theta^*|\theta)$ and decide whether to accept it based on the ratio of the target distribution at θ^* and the target distribution at θ.

[1] Wikipedia: What is the Stationary distribution in a Markov chain?

Algorithm 1 Metropolis-Hastings (MH)

1. **Require:** Target distribution $\pi(\theta)$ up to a constant, proposal distribution $q(\theta^*|\theta^{(t-1)})$, initial position $\theta^{(0)}$, total number of iterations N
2. **For** $t = 1, 2, \ldots, N$ **do**
   2.1. Sample proposal $\theta^* \sim q(\theta^*|\theta^{(t-1)})$
   2.2. Compute acceptance ratio:
$$\alpha(\theta^{(t-1)}, \theta^*) \leftarrow \min\left(1, \frac{\pi(\theta^*)q(\theta^{(t-1)}|\theta^*)}{\pi(\theta^{(t-1)})q(\theta^*|\theta^{(t-1)})}\right)$$
   2.3. Draw $u \sim \text{Uniform}(0, 1)$
   2.4. **If** $u < \alpha(\theta^{(t-1)}, \theta^*)$ **then**
      2.4.1. Accept: $\theta^{(t)} \leftarrow \theta^*$
   2.5. **Else**
      2.5.1. Reject: $\theta^{(t)} \leftarrow \theta^{(t-1)}$
   2.6. **End if**
3. **End for**
4. **Return** $\{\theta^{(1)}, \theta^{(2)}, \ldots, \theta^{(N)}\}$ % Return sequence of samples

Theory Question 10. Prove that the Metropolis-Hastings algorithm satisfies the detailed balance condition:
$$\mathbb{P}(\theta)q(\theta^*|\theta)\alpha(\theta, \theta^*) = \mathbb{P}(\theta^*)q(\theta|\theta^*)\alpha(\theta^*, \theta).$$
Discuss the implications of detailed balance for the convergence of the Markov chain to the target distribution.

Theory Question 11. In high-dimensional settings, the performance of MCMC methods can deteriorate due to the "curse of dimensionality." Derive an expression for the effective sample size in the context of high-dimensional MCMC sampling, and show how the auto-correlation time of the samples increases with dimensionality. Can you suggest some strategies to mitigate these issues?

8 Hamiltonian Monte Carlo

In Bayesian Neural Networks (BNNs), the posterior distribution over weights, $p(\boldsymbol{\theta}|\mathcal{D})$, is high-dimensional and often exhibits complex correlations due to the non-linear nature of neural architectures. As you saw the Metropolis-Hastings MCMC method, struggles to efficiently sample from such distributions, as random-walk proposals lead to slow exploration and high autocorrelation. How can we sample the posterior more effectively to capture uncertainty in BNN weights?

Hamiltonian Monte Carlo (HMC) addresses this challenge by leveraging **gradient information** to propose samples, guiding exploration of the posterior using principles from Hamiltonian dynamics. Here the Markov chain from which we sample in BNNs is produced analogous to paths of particles using these dynamics.

8.1 Definitions

Base Problem: Generate samples $\{\boldsymbol{\theta}^{(0)}, \boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(N)}\}$ from the posterior distribution $p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ represents BNN weights, $\mathcal{D}$ is the observed data, $p(\mathcal{D}|\boldsymbol{\theta})$ is the likelihood, and $p(\boldsymbol{\theta})$ is the prior (e.g., Gaussian).

HMC treats BNN weights $\boldsymbol{\theta}$ as positions in a physical system, with auxiliary momentum variables $\boldsymbol{\rho}$ introduced to simulate dynamic trajectories. These trajectories efficiently traverse the high-dimensional posterior, enabling robust uncertainty quantification in predictions, which is critical for applications like medical diagnostics or autonomous systems.

The momentum variable $\boldsymbol{\rho}$ is typically drawn from a Gaussian distribution $p(\boldsymbol{\rho}) = \mathcal{N}(\mathbf{0}, \mathbf{M})$, where $\mathbf{M}$ is a mass matrix. The posterior defines a potential energy:

$$U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta}|\mathcal{D}) = -\log p(\mathcal{D}|\boldsymbol{\theta}) - \log p(\boldsymbol{\theta}), \tag{21}$$

reflecting the negative log-posterior, which is high when $\boldsymbol{\theta}$ is unlikely under the data and prior. The kinetic energy is:

$$K(\boldsymbol{\rho}) = \frac{1}{2}\boldsymbol{\rho}^T\mathbf{M}^{-1}\boldsymbol{\rho}, \tag{22}$$

corresponding to the negative log-density of $p(\boldsymbol{\rho})$. The Hamiltonian combines these:

$$H(\boldsymbol{\theta}, \boldsymbol{\rho}) = U(\boldsymbol{\theta}) + K(\boldsymbol{\rho}). \tag{23}$$

The joint distribution is:

$$p(\boldsymbol{\theta}, \boldsymbol{\rho}) = e^{-H(\boldsymbol{\theta},\boldsymbol{\rho})}/Z, \tag{24}$$

where Z is a normalizing constant. Since $\boldsymbol{\theta}$ and $\boldsymbol{\rho}$ are independent ($p(\boldsymbol{\rho})$ does not depend on $\boldsymbol{\theta}$), marginalizing over $\boldsymbol{\rho}$ recovers the target posterior:

$$\int p(\boldsymbol{\theta}, \boldsymbol{\rho}) d\boldsymbol{\rho} \propto p(\boldsymbol{\theta}|\mathcal{D}). \tag{25}$$

Note: we have a small abuse of notation here as $p(\theta, \rho)$ is actually $p(\theta, \rho|\mathcal{D})$. Also, the Gaussian assumptions are not necessary for the theory of HMC to be valid in general.

8.2 A Little Bit of Physics

The Hamiltonian $H(\boldsymbol{\theta}, \boldsymbol{\rho})$ represents the total energy, **conserved** along a trajectory. This means the Hamiltonian should not change with time. Regarding that, the dynamics of $\boldsymbol{\theta}$ and $\boldsymbol{\rho}$ are governed by Hamilton's equations (optionally, check how these keep the conservation):

$$\frac{d\boldsymbol{\theta}}{dt} = \frac{\partial H}{\partial \boldsymbol{\rho}} = \frac{\partial K(\boldsymbol{\rho})}{\partial \boldsymbol{\rho}} \overset{\text{from } 22}{=} \mathbf{M}^{-1}\boldsymbol{\rho}, \tag{26}$$

$$\frac{d\boldsymbol{\rho}}{dt} = -\frac{\partial H}{\partial \boldsymbol{\theta}} = -\frac{\partial U(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \overset{\text{from } 21}{=} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}|\mathcal{D}). \tag{27}$$

The gradient $\nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}|\mathcal{D}) = \nabla_{\boldsymbol{\theta}} \log p(\mathcal{D}|\boldsymbol{\theta}) + \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta})$ drives the momentum update, steering the trajectory toward regions of high posterior density. In BNNs, these gradients are computable because neural network likelihoods are differentiable, making HMC particularly suitable.

8.3 Discretizing Hamiltonian Equations

Exact Hamiltonian dynamics are continuous, but HMC approximates them numerically using the **leapfrog integrator**, a symplectic method that preserves the Hamiltonian approximately. To propose a new state $(\boldsymbol{\theta}^*, \boldsymbol{\rho}^*)$, HMC simulates a trajectory over L steps with step size ϵ:

1. **Half-step momentum update**:

$$\boldsymbol{\rho} \leftarrow \boldsymbol{\rho} + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}|\mathcal{D}), \tag{28}$$

   using the posterior gradient to adjust momentum based on the current weights' likelihood and prior.

2. **Full-step position update**:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \mathbf{M}^{-1} \boldsymbol{\rho}, \tag{29}$$

   moving the weights according to the momentum, analogous to a particle's motion.

3. **Second half-step momentum update**:

$$\boldsymbol{\rho} \leftarrow \boldsymbol{\rho} + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}|\mathcal{D}), \tag{30}$$

   updating momentum using the new weights' gradient.

These steps are repeated L times to generate a proposal $(\boldsymbol{\theta}^*, \boldsymbol{\rho}^*)$, which is accepted or rejected via a Metropolis step to ensure the chain targets $p(\boldsymbol{\theta}|\mathcal{D})$. In BNNs, the leapfrog integrator leverages the differentiability of $p(\mathcal{D}|\boldsymbol{\theta})$, enabling efficient traversal of the weight posterior. Backpropagation computes these gradients efficiently for the neural network's output with respect to its weights. The choice of ϵ and L balances accuracy and computational cost, critical for scaling HMC to large neural networks.

8.4 HMC Algorithm

Algorithm 2 Hamiltonian Monte Carlo for BNNs

Require: Initial weights $\boldsymbol{\theta}^{(0)}$, step size ϵ, number of leapfrog steps L, mass matrix $\mathbf{M}$, number of samples N

Set $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^{(0)}$
for $t = 1$ to N **do**
  Sample momentum $\boldsymbol{\rho} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$
  for $l = 1$ to L **do**
    $\boldsymbol{\rho}^* \leftarrow \boldsymbol{\rho}^* + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}^*} \log p(\boldsymbol{\theta}^* | \mathcal{D})$ {Half-step momentum}
    $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}^* + \epsilon \mathbf{M}^{-1} \boldsymbol{\rho}^*$ {Full-step position}
    $\boldsymbol{\rho}^* \leftarrow \boldsymbol{\rho}^* + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}^*} \log p(\boldsymbol{\theta}^* | \mathcal{D})$ {Half-step momentum}
  end for
  Compute acceptance probability $\alpha = \min\left(1, \exp\left(H(\boldsymbol{\theta}, \boldsymbol{\rho}) - H(\boldsymbol{\theta}^*, \boldsymbol{\rho}^*)\right)\right)$
  Draw $u \sim \text{Uniform}(0, 1)$
  if $u < \alpha$ **then**
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^*$
  end if
  Store sample $\boldsymbol{\theta}^{(t)} \leftarrow \boldsymbol{\theta}$
end for
return Samples $\{\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(N)}\}$

8.5 Questions

Theory Question 12. Checking Conditions for MCMC

1. Prove that the leapfrog integrator, denoted as $\Omega_\epsilon : (\boldsymbol{\theta}, \boldsymbol{\rho}) \to (\boldsymbol{\theta}', \boldsymbol{\rho}')$, is reversible. Specifically, show that applying the leapfrog step to $(\boldsymbol{\theta}', -\boldsymbol{\rho}')$ returns the original state $(\boldsymbol{\theta}, -\boldsymbol{\rho})$.
2. Prove that the leapfrog integrator preserves the volume in phase space, i.e., the determinant of the Jacobian of the transformation Ω_ϵ is 1. Assume $\mathbf{M} = \mathbf{I}$ for simplicity.
3. Show how parts 1 and 2 lead to confirming that the detailed balance condition is satisfied
4. Argue about HMC's Markov chain property, Whether it is irreducible or not, and its aperiodicity.

Theory Question 13. Let's explore a bit more:

1. In the HMC algorithm, neither the detailed balance condition nor the (approximate) conservation of the Hamiltonian is violated even if different step sizes are used for different variables. Why? Give an example distribution where this is useful.
2. In the HMC algorithm, the detailed balance condition is not broken even if we use different distributions $p(\boldsymbol{\theta}|\mathcal{D})$ for the leapfrog time evolution and the Metropolis test. Which one will be the distribution obtained in this way? Why? What is the advantage and disadvantage of using such a trick? (hint: consider $e^{-\frac{1}{x^2} - x^2}$)

Good luck!